Movie Recommendation System

Using Collaborative Filtering

Project Report

Submitted by: Anvitha Prasad
Institution: Cognevance Technologies

Date: November 8, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Dataset **2**

4 Methodology and Workflow **2**
4.1 Data Cleaning and Preprocessing 3
4.2 Exploratory Analysis 3

5 Recommendation Algorithm **3**

6 Data Analysis and Visualizations **4**
6.1 Rating Distribution 4
6.2 Most Popular Movies 4
6.3 Highest-Rated Movies 5
6.4 Genre Analysis 5
6.5 Trends Over Time 6

7 Recommendation Output **7**
7.1 Model Evaluation 7

8 Limitations **8**

9 Future Work **8**

10 Conclusion **8**

1 Introduction

Recommendation systems help users discover items they are likely to enjoy from a very large catalogue. Streaming platforms, online stores and music services all rely on them to improve user experience and engagement. This project builds a simple AI-based movie recommender that suggests movies to users based on their past ratings and the preferences of similar users.

2 Objectives

- Collect and clean a movie ratings dataset.
- Analyze user ratings and movie genres.
- Implement user-based collaborative filtering.
- Generate personalized recommendations for different users.
- Visualize popular movies and rating trends.

3 Dataset

The project uses the **MovieLens Latest Small** dataset published by GroupLens Research (https://grouplens.org/datasets/movielens/latest/). It contains about 100,000 ratings from roughly 600 users on about 9,000 movies.

Table 1: Dataset files and columns

File	Columns	Description
ratings.csv	userId, movieId, rating, timestamp	User ratings (0.5 to 5.0)
movies.csv	movieId, title, genres	Movie title with release year and genres

4 Methodology and Workflow

The project follows the pipeline shown in Figure 1.

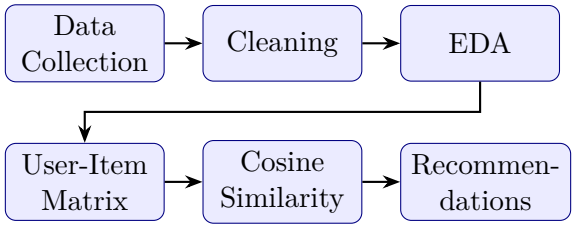


Figure 1: Project workflow

4.1 Data Cleaning and Preprocessing

Missing values and duplicates were removed. The Unix timestamp was converted to a date, the release year was extracted from the movie title, and the pipe-separated genres were split into lists. The ratings and movies tables were merged on `movieId` for analysis.

4.2 Exploratory Analysis

Movie-level statistics (average rating and number of ratings) and genre-level statistics were computed. For the "top rated" list, only movies with at least 50 ratings were considered, because movies with very few ratings can have unreliable averages.

5 Recommendation Algorithm

The system uses **user-based collaborative filtering**. The idea is that users who rated movies similarly in the past will like similar movies in the future.

1. Build a user-item matrix with users as rows and movies as columns.
2. Mean-center each user's ratings to remove the difference between generous and strict raters.
3. Compute the cosine similarity between every pair of users:

$$\text{sim}(u, v) = \frac{\mathbf{r}_u \cdot \mathbf{r}_v}{\mathbf{r}_u \, \mathbf{r}_v}$$

4. Select the k most similar users (neighbors) for the target user.
5. Predict the rating of an unseen movie i for user u:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) \, (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} \text{sim}(u, v)}$$

6. Recommend the top n movies with the highest predicted rating that the user has not yet seen and that were rated by at least three neighbors.

Listing 1: Core recommendation function (simplified)

```
def recommend(user_id, n=10, k=30, min_neighbors=3):
    sims = user_sim[user_id].drop(user_id).nlargest(k)
    neighbor_ratings = centered.loc[sims.index]
    scores = neighbor_ratings.T.dot(sims) / sims.abs().sum()
    scores = scores + user_means[user_id]
    seen = user_item.loc[user_id].dropna().index
    scores = scores.drop(seen, errors="ignore")
    ...
```

6 Data Analysis and Visualizations

6.1 Rating Distribution

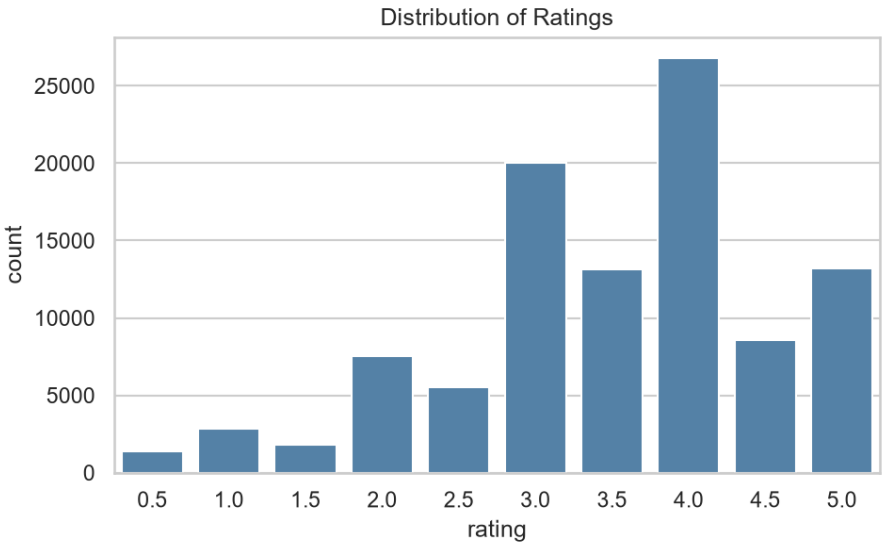


Figure 2: Distribution of ratings

Most ratings fall between 3.0 and 4.0, with 4.0 being the most common value. *(Edit this sentence to match your own chart.)*

6.2 Most Popular Movies

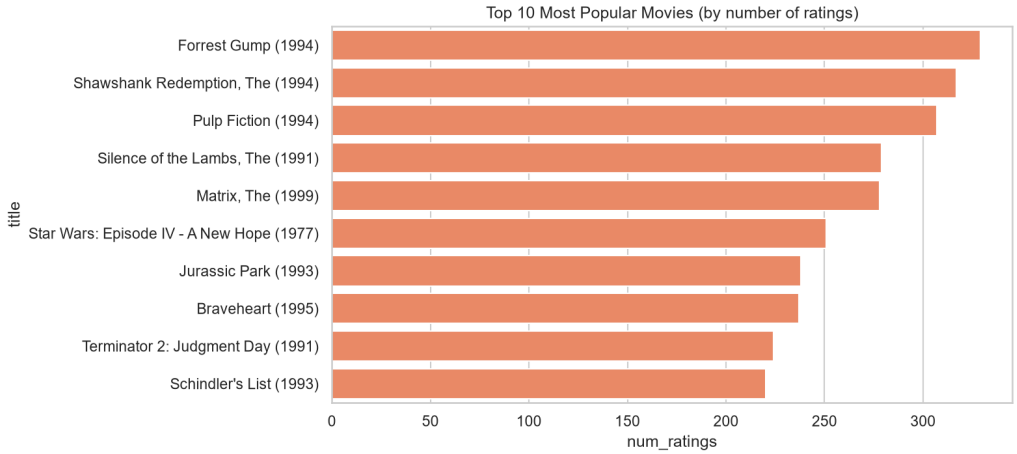


Figure 3: Top 10 movies by number of ratings

6.3 Highest-Rated Movies

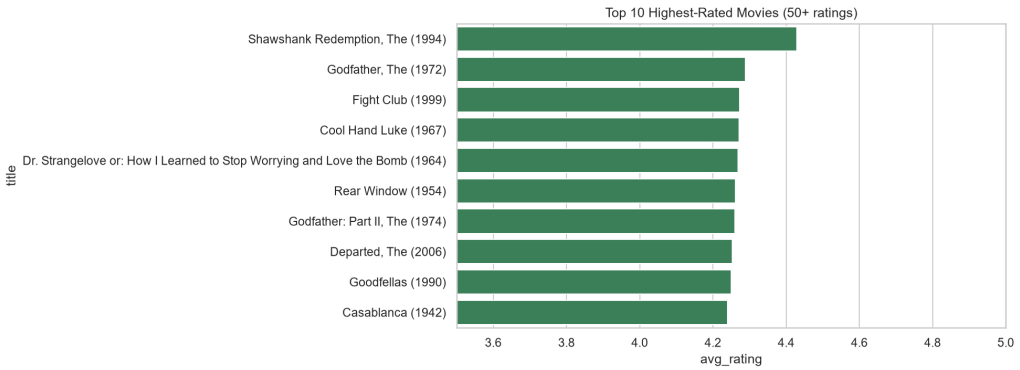


Figure 4: Top 10 highest-rated movies (at least 50 ratings)

6.4 Genre Analysis

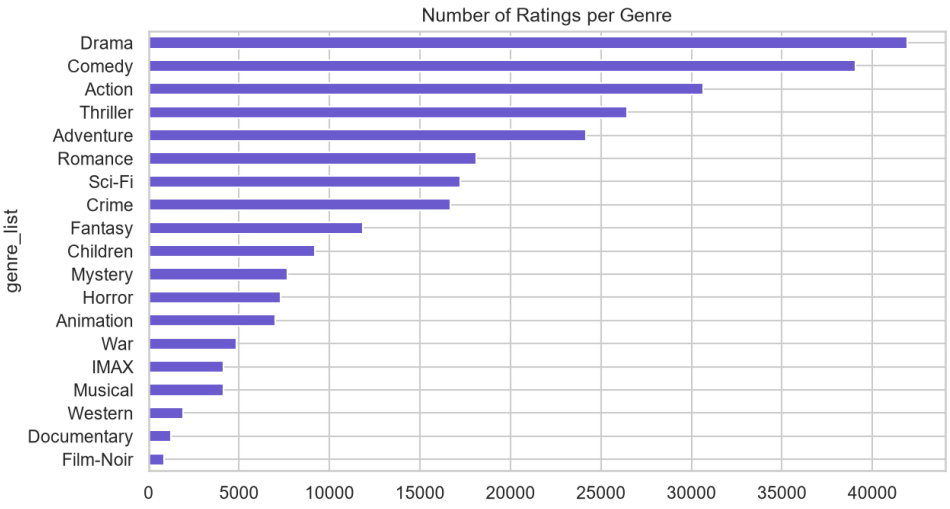


Figure 5: Number of ratings per genre

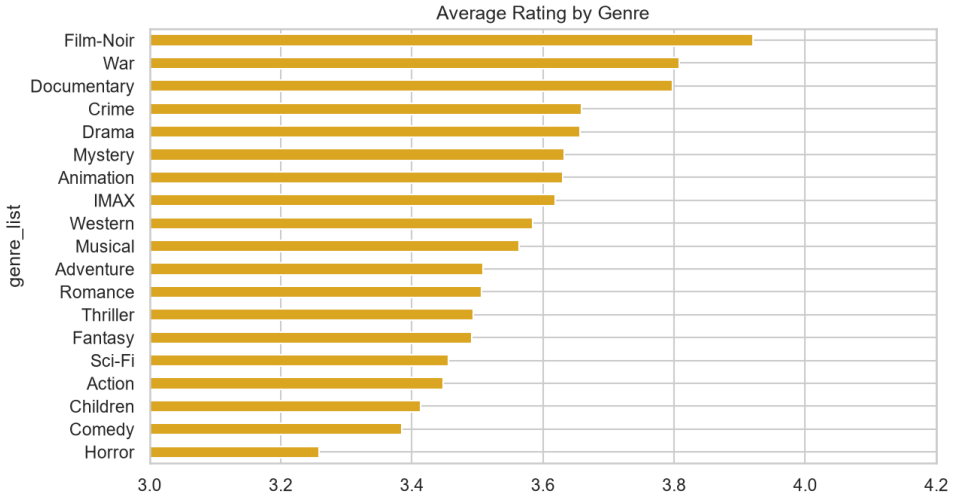


Figure 6: Average rating by genre

Drama and Comedy are the most frequently rated genres, while genres such as Film-Noir and War tend to receive higher average ratings. *(Verify against your charts.)*

6.5 Trends Over Time

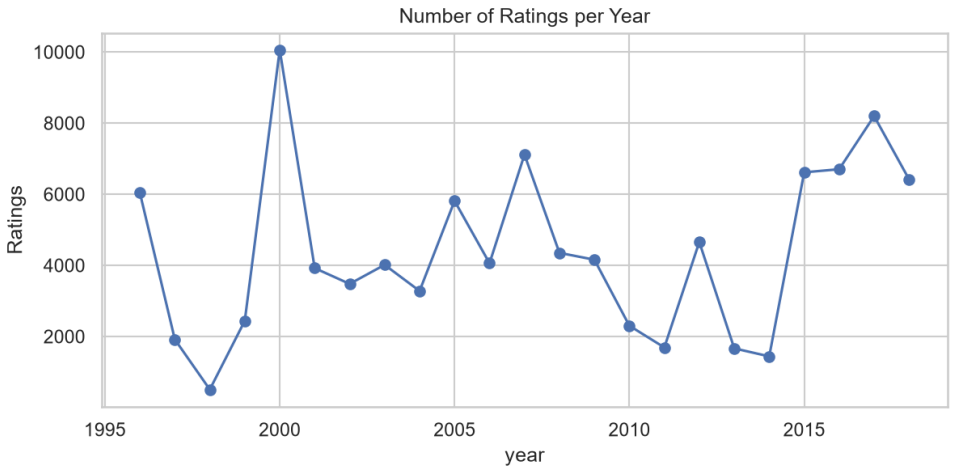


Figure 7: Number of ratings per year

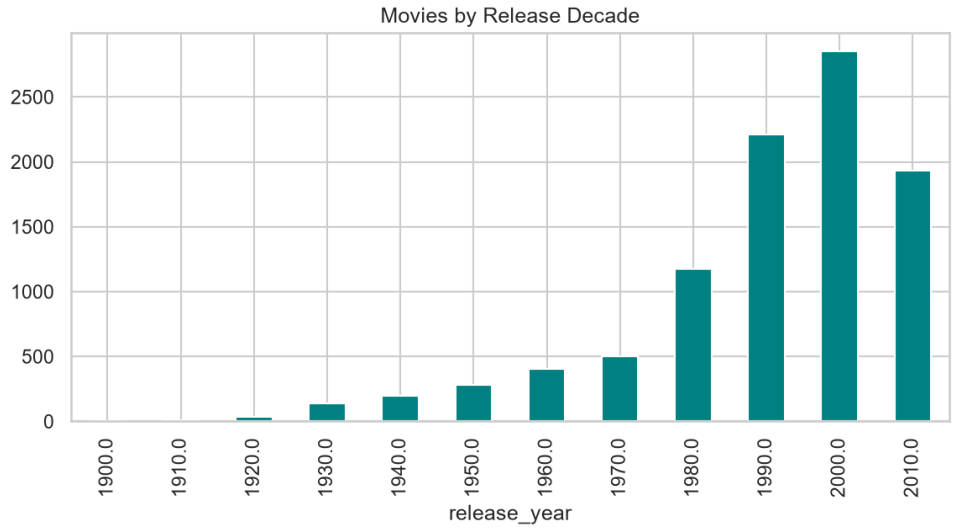


Figure 8: Movies by release decade

7 Recommendation Output

Table 2 shows an example of the top recommendations generated for one user. Replace the rows with your own output from `outputs/recommendations.csv`.

Table 2: Sample recommendations for User 1

Rank	Movie	Predicted Rating
1	Movie Title A (Year)	4.85
2	Movie Title B (Year)	4.78
3	Movie Title C (Year)	4.71
4	Movie Title D (Year)	4.66
5	Movie Title E (Year)	4.60

Recommendations for the other sample users (50, 150, 300 and 500) are available in the file `outputs/recommendations.csv` in the repository.

7.1 Model Evaluation

The model was evaluated by holding out 20% of the ratings as a test set and predicting a random sample of them. The resulting root mean squared error was **RMSE = X.XXX** (replace with your value), where

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(r_i - \hat{r}_i)^2}.$$

8 Limitations

- **Cold start:** new users or new movies have no ratings, so recommendations cannot be generated for them.
- **Sparsity:** most users rate only a small fraction of all movies.
- **No content information:** genres and other movie features are not used by the algorithm.
- **Scalability:** computing a full user-user similarity matrix becomes expensive for very large datasets.

9 Future Work

- Item-based collaborative filtering for comparison.
- Matrix factorization methods such as SVD.
- A hybrid model that combines collaborative and content-based filtering.
- A web interface built with Streamlit.

10 Conclusion

This project demonstrated the complete workflow of a recommender system: collecting data, cleaning it, analyzing user behavior, building a collaborative filtering model and presenting personalized recommendations with supporting visualizations. Although simple, the approach provides a solid foundation for more advanced recommendation techniques.

References

1. F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, 2015.
2. Scikit-learn documentation: `https://scikit-learn.org`
3. Pandas documentation: `https://pandas.pydata.org`
4. Project repository: `https://github.com/anveez/cognevance_movie-recommendation-system`